

From Abandoned carts to recovered revenue:

Building Real-Time AI Agents with AWS & Elasticsearch

Event-driven intelligence using EventBridge, Lambda, MCP & Elastic AI Agent



EventBridge · Lambda · SES · S3 · API Gateway | Elasticsearch · Elastic AI Agent · MCP Tools

The Abandoned Cart \$4.6 Trillion Problem

\$4.6T

lost annually to abandoned carts
globally

70%

average cart abandonment rate across
e-commerce

1-size

fits all — the flawed approach most businesses
use

✗ The Old Way — Generic Discount Blast

- VIP customers get the same treatment as first-time shoppers
- Payment failures get discounts — when they need a **payment retry**
- High fraud-risk customers receive financial incentives they should be blocked from
- Shipping concerns get irrelevant promotional offers

✓ The Right Way — Context-Aware

- **Recovery** Diagnose **why** the cart was abandoned before acting Match the right recovery action to the
- right root cause
- Respect customer segment — VIPs, standard, high-risk all treated differently Close the feedback loop — track outcomes, prevent duplicates

The solution? Intelligent, context-aware recovery using **Elastic AI Agent + AWS** — that automatically diagnoses abandonment reasons and selects the optimal recovery strategy per cart.

What We'll Cover Today

01

The Abandoned Cart Problem
Why intelligent recovery matters

02

System Architecture
AWS & Elastic — 3 layers working together

03

Data Ingestion & Elasticsearch Layer
From raw events to structured state & diagnosis

04

Detection, Diagnosis & Orchestration
The Elastic scheduled workflow in action

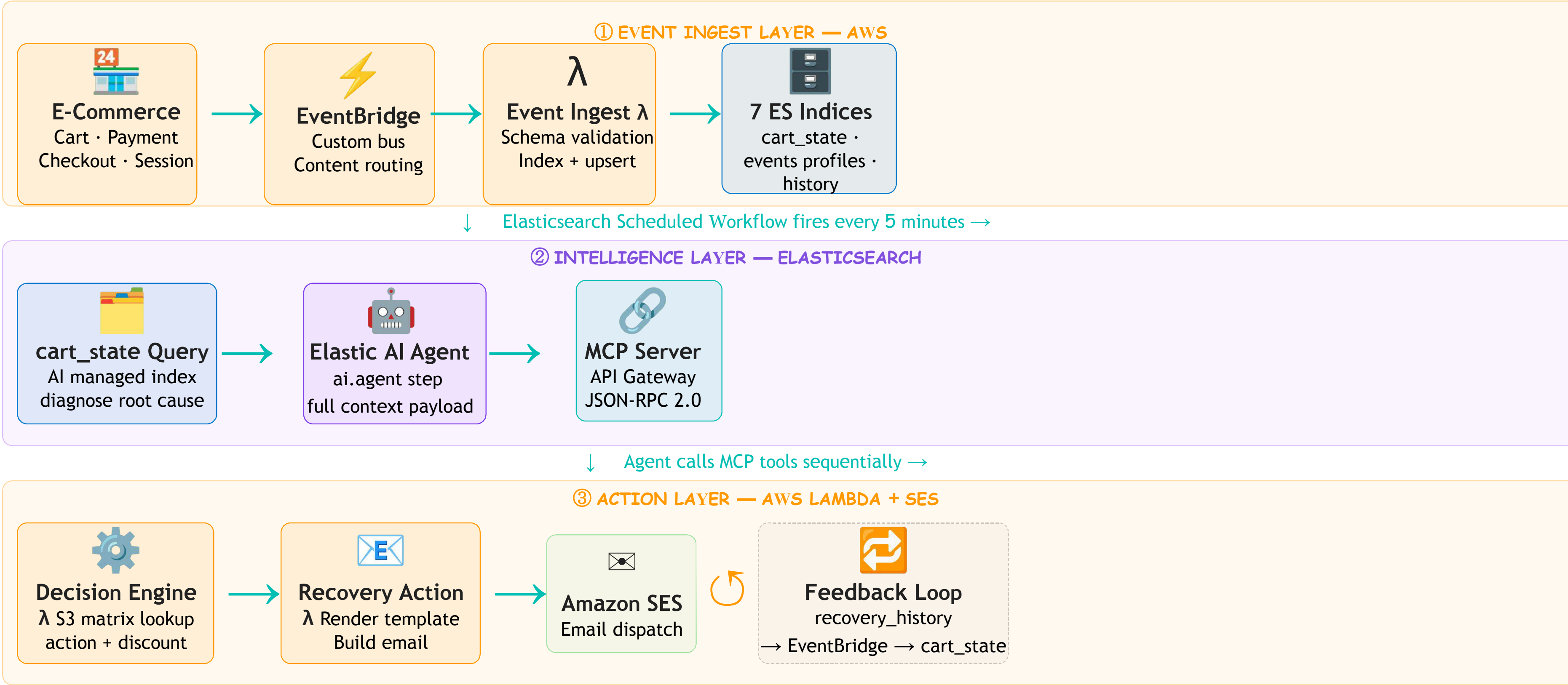
05

Elastic AI Agent + MCP Tools
How Lambda becomes AI-native via MCP protocol

06

Key Takeaways & Lessons Learnt
Patterns & hard lessons to take home

High-Level System Architecture — 3 Clear Layers



Amazon EventBridge + Lambda — Event Ingestion



Routing the Data Flow

All incoming events (like adding items to cart, entering checkout, or payment failures) hit an EventBridge custom bus. Powerful content-based routing rules instantly filter the data, triggering specific Lambdas matching our exact criteria.

Event-Driven Compute

The Event Ingest Lambda wakes up immediately to process filtered events. It maps them efficiently to their respective Elasticsearch index (like ``payment_logs`` or ``cart_events``), skipping heavy API overhead or slow polling logic entirely.



Managing the ``cart_state``

Crucially, it maintains a single source of truth: the AI-managed ``cart_state``. Whenever an event is processed, it transitions the cart's status from ``active`` → ``completed`` → ``recovery_sent``. This allows our intelligence layer to always know a cart's accurate state without querying endless raw logs!

7 Indices — One Source of Truth



cart_events

cart_id, customer_id, session_id, cart_value,
device_type, referrer



checkout_events

step, status, shipping_cost, payment_method,
total, currency



payment_logs

status, failure_code, failure_message,
retryable, gateway_latency_ms



customer_profiles

segment (VIP/standard/default), fraud_risk,
lifetime_value, preferred_channel



session_metrics

p95_latency_ms, error_rate, page_views,
device_type, browser



recovery_history

action_taken, template_used, outcome,
timestamp



cart_state ← AI-Managed

active → **completed** → **recovery_sent**

Lifecycle managed by Event Ingest Lambda. Drives
the detection workflow and prevents duplicates.

cart_state is the intelligence layer — it's not a raw event log, it's a **derived, maintained state machine** that the workflow queries every 5 minutes.

Elastic as Orchestrator

A single scheduled **Elasticsearch workflow** runs every 5 minutes — no Lambda, no CloudWatch rule — and drives the entire detection-to-recovery pipeline.



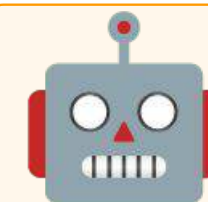
① Detect Abandoned Carts

Query `cart_state` index for carts with `status=active` and `check_at < now`. Up to 100 carts per cycle. No external trigger needed — fully internal to Elasticsearch.



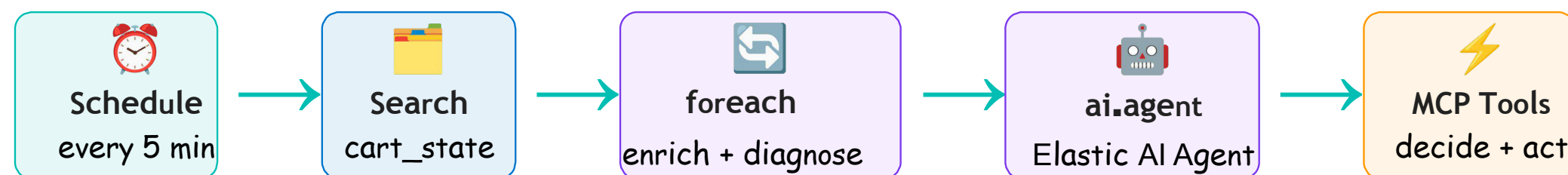
② Find Root Cause

For each cart, the workflow fetches from **5 indices**: `customer_profiles`, `cart_events`, `checkout_events`, `payment_logs`, `session_metrics`. Evaluates 5 diagnostic branches in priority order to determine *Why* the cart was abandoned.

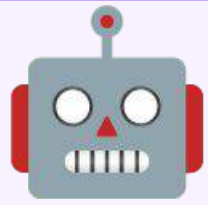


③ Hand Off to AI Agent

Emits a structured JSON diagnosis payload via `data.set` — containing `root_cause`, `segment`, `cart_value`, `fraud_risk` and more — directly into an `ai.agent` step. The agent decides and acts autonomously.



Elastic **AI Agent** — The Decision Maker



What is the Elastic AI Agent?

A native **Agent Builder** in Elasticsearch. Configured with an LLM inference endpoint (OpenAI / Azure / self-hosted). Receives a structured JSON context payload and autonomously chains tool calls to resolve the task.

Activated inside a workflow via the `ai.agent` step type — no external orchestration needed.



What It Does

1. Reads the structured diagnosis
2. Calls `decision_engine` MCP tool → gets recommended action, discount, template
3. If action $\neq$ blocked, calls `recovery_action` MCP tool → sends email, logs outcome

No hardcoded if/else — the agent reasons over context and decides the sequence autonomously.



Key Properties

`ai.agent` step type

`tools /list` — discovery

`tools/call` — execution

Connects to MCP Server over **Streamable HTTP (JSON-RPC 2.0)** secured with an API Gateway API key.



What It Receives

The full diagnosis payload from the workflow:

```
cart_id • customer_id •
root_cause cart_value •
user_segment • fraud_risk
customer_profile • session_metrics
```


MCP — Making Lambda AI-Native



What is MCP?

Model Context Protocol is an open standard that lets AI agents discover and call external tools via a standardised **JSON-RPC 2.0** interface over HTTP.

Think of it as **OpenAPI for AI Agents** — the agent asks "what tools exist?" and the server responds with schemas.



Lambda as an MCP Tool

API Gateway exposes `POST /mcp` → MCP Server Lambda routes JSON-RPC calls → invokes Decision Engine or Recovery Action Lambda.

The AI Agent **doesn't know it's calling Lambda** — it just sees a tool named `decision_engine`.

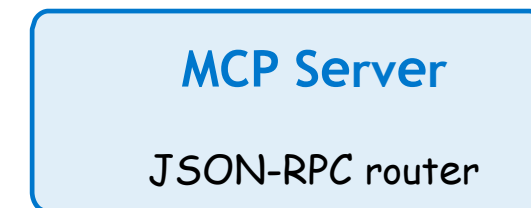
LAMBDA ARCHITECTURE



↓ JSON-RPC 2.0 over HTTPS

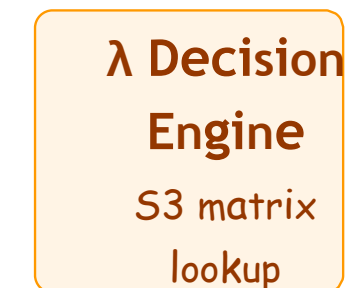


↓ routes calls λ



↓ InvokeFunction

↓ InvokeFunction



Two MCP Tools — Sequential Chain



Tool 1: decision_engine

Input: cart_id, customer_id, segment, abandonment_reason, cart_value

Logic:

1. Load decision-matrix.json from S3
2. Normalise segment key → look up rule
3. Check high-cart-value override (VIP>\$500 → 15%, Std>\$300 → 10%)
4. Apply fraud guardrail — block if high_fraud_risk + payment_failure

Output: action type, discount %, free_shipping, message



Tool 2: recovery_action

Input: cart_id, customer_id, email, action + decision from Tool 1

Logic:

1. Build personalised HTML email (discount badge, CTA button)
2. Send via Amazon SES
3. Publish `recovery_history` event → EventBridge
4. Event Ingest Lambda re-indexes → cart_state → `recovery_sent`

Output: status, channel, message_id, sent_at

The AI Agent **autonomously decides the order and parameters** for both tool calls. It receives the diagnosis payload once and chains the two MCP calls — no hardcoded orchestration.

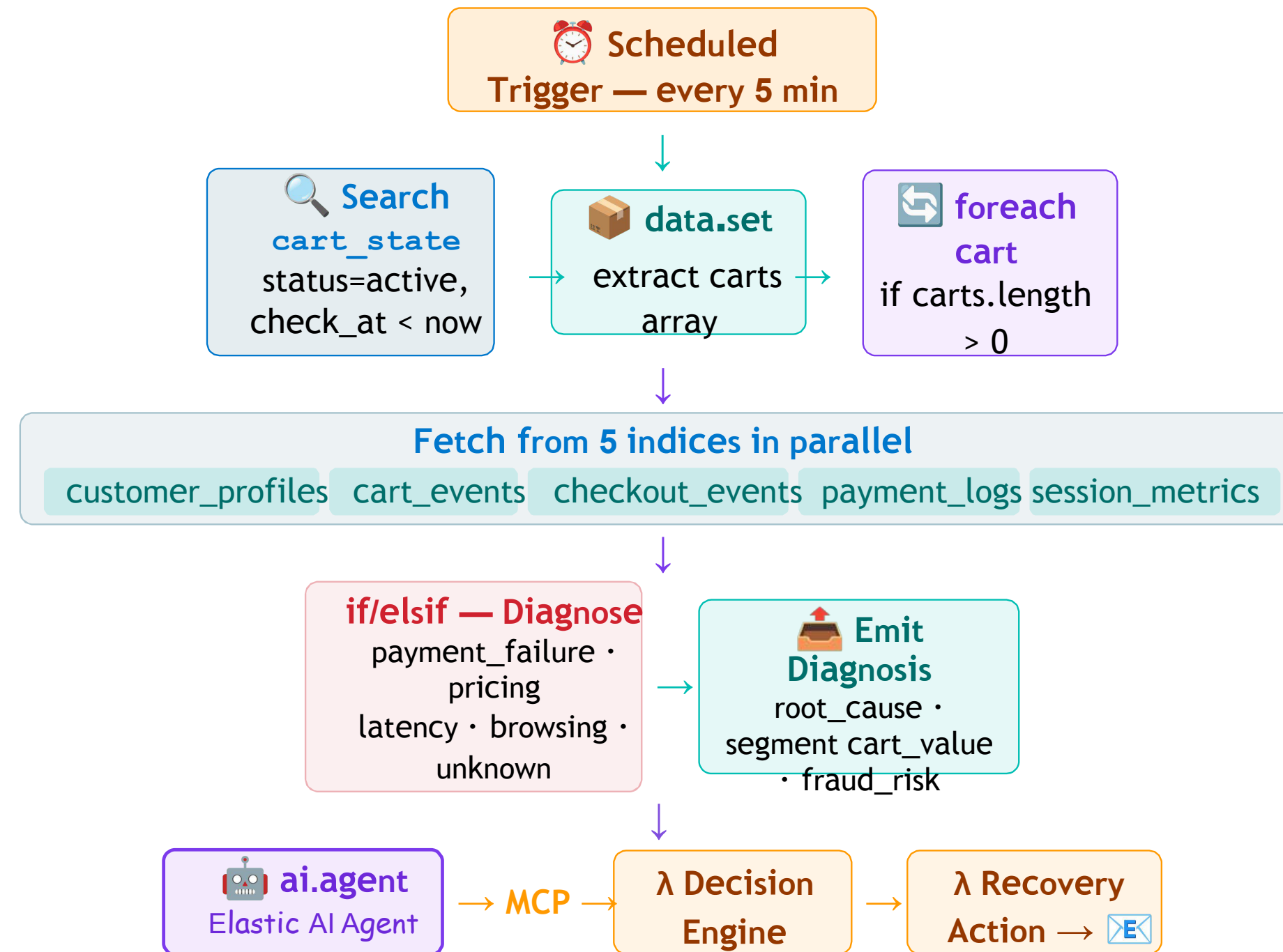
The Decision Matrix — Rules in S3

Segment	Root Cause	Action	Incentive	Est. Success
VIP	payment_failure	payment_retry	—	85%
VIP	high_cart_value (> \$500)	discount	15% off	78%
VIP	shipping_issue	free_shipping	Free ship	72%
standard	high_cart_value (> \$300)	discount	10% off	65%
standard	browsing_abandonment	reminder	—	40%
high_fraud_risk	payment_failure	blocked	None	0%
high_fraud_risk	any other reason	reminder_only	None	—



Matrix lives in **S3** as JSON. Update business rules without redeploying any Lambda — just update the file. Lambda caches it in memory on cold start.

Detection **Workflow** — End-to-End Pipeline



The **AWS Stack** — Fully Serverless



Amazon EventBridge

Custom event bus — all 7 event types flow through here. Batch of 10 per PutEvents call. EventBridge rule triggers Event Ingest Lambda on every event.



API Gateway

POST /mcp endpoint with API key auth (x-api-key header). Usage plan: 100 req/s, 10,000 req/day. This is the MCP Server's public URL.



4× AWS Lambda Functions

event_ingest — indexes events + manages
cart_state mcp_server — JSON-RPC 2.0 router
decision_engine — S3 matrix lookup
recovery_action — SES + EventBridge publish



Amazon S3 + SES

S3 stores decision-matrix.json — no DB needed for rules. SES sends branded HTML emails with discount badges, CTA buttons, and dynamic subject lines.

CloudFormation / SAM

One-command deploy: ./deploy.sh dev

IAM least-privilege roles

The Elastic Stack — Intelligence Layer



7 Elasticsearch Indices

Append-only logs: `cart_events` · `checkout_events` · `payment_logs` · `session_metrics`

Derived state: `cart_state` — the workflow's single detection target

Feedback + profiles: `recovery_history` · `customer_profiles`

Strict mappings

dynamic: strict

Schema-first



Scheduled Workflows — Native Orchestration

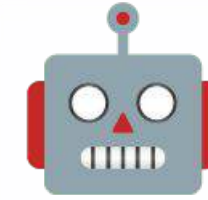
YAML-defined pipelines running inside the cluster. Step types:

`elasticsearch.search` · `foreach` · `if/elsif` · `data.set` · `ai.agent`

No external cron

Liquid templating

Cross-index joins



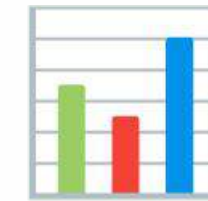
Elastic AI Agent

Native `ai.agent` step type in workflows. Configured with an LLM inference endpoint (OpenAI / Azure / self-hosted). Receives structured JSON context, calls MCP tools autonomously.

tools/list — discovery

tools/call — execution

Conditional chaining



Kibana — Full Observability

Recovery funnel dashboards, diagnosis breakdown by root cause, segment performance — all on the same cluster. Workflow execution history, agent call logs, and `cart_state` transitions queryable live in Dev Tools.

Zero export needed

Real-time

Cross-index DSL

If MCP tools decide everything — what does the Agent actually do?

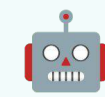


Without Agent — Plain API Chaining

You write code like:

```
result = decision_engine(segment, reason, cart_value)
recovery_action(result.action, email, cart_id)
```

- ✗ Field mapping is hardcoded — *you* wrote it
- ✗ Order of calls is hardcoded — no adaptation
- ✗ If `action = blocked`, you need an `if` to skip email
- ✗ Add a 3rd tool → re-deploy and re-wire everything
- ✗ No reasoning — input must always be clean & complete



With Elastic AI Agent

You give the agent the full diagnosis payload and say:

"Decide and execute the best recovery action."

- ✓ Agent **extracts** parameters from rich JSON context
- ✓ Agent **decides** the call order and whether to call at all
- ✓ If `blocked` → agent skips recovery without explicit code
- ✓ New MCP tool added → agent discovers and uses it automatically
- ✓ Handles nulls, missing fields, unexpected states gracefully



Intelligence #1 — Context Reading

The diagnosis payload has 15+ fields from 5 indices. The agent reads the *whole picture* — customer segment, fraud risk, cart value, device type, session latency — and maps only the relevant fields to each tool call. An API integration maps exactly what you coded, nothing more.



Intelligence #2 — Conditional Chaining

The agent reads the *output* of `decision_engine` before calling `recovery_action`. If `action = blocked`, it understands no email should be sent and returns a summary without calling the second tool. No `if/else` in your codebase.



Intelligence #3 — Tool Discovery

The agent calls `tools/list` first — it *discovers* available tools and their schemas at runtime. Add a `sms_action` MCP tool tomorrow and the agent will consider using it without any code change. Pure API chaining can never do this.

The **decision matrix** answers **WHAT** to do. The AI Agent answers **HOW** to act on that — **adaptively, conditionally, and extensibly** — across any number of tools, without a developer re-wiring every edge case.

5 Patterns to Take Home

1 Derive State — Don't Query Raw Events

Maintain a derived state index (`cart_state`) updated at ingestion time. Decision queries become trivial single-index lookups, idempotency is enforced at the index level, and you never process the same record twice.

2 Search Engine as Orchestrator

Elasticsearch's scheduled workflows (`foreach`, `if/elsif`, `data.set`, `ai.agent`) can replace Airflow or Step Functions for data-centric pipelines. If your decisions are driven by data already in the search cluster, run the logic *inside* the cluster.

3 MCP makes AI agents composable

Expose business logic as typed MCP tools via API Gateway + Lambda. Any AI model — Elastic Agent, Claude, GPT — can discover and call the same tools without custom wiring. Adding a new capability means adding a new tool, not redeploying the agent.

4 Config-as-Data on S3 Beats Rules in Code

Business rules that change regularly (discount thresholds, segment policies) belong in S3 as JSON — not in Lambda code. No deployment, no code review, no downtime. Non-engineers can own the rules.

5 Design the Feedback Loop First

If the feedback path (action taken → outcome logged → state updated) is a first-class design decision from day one, the entire system becomes self-correcting and far easier to debug and evaluate. Don't retrofit it.

Lessons Learnt



Liquid Templating is Powerful but Unforgiving

No syntax highlighting, no step-through debugger. Writing complex `if/elseif` chains with nested object access requires extreme care. Validate the JSON output via `recovery_history` entries.



EventBridge Batching Has Sharp Limits

`PutEvents` accepts at most 10 entries per call. Production systems emitting hundreds of events per second need a queue or batching layer in front of EventBridge.



Define Index Mappings First

Strict schemas upfront prevent downstream data type errors. Retrofitting mappings after ingestion has started means re-indexing — not fun.

Test Decision Matrix Edge Cases Early

The matrix has three dimensions (segment × reason × cart value) with fallback paths. A missing rule silently falls back to "reminder" — which looks correct in logs but masks the real gap.



Cold-Start Chains Amplify Latency

MCP Server → Decision Engine → Recovery Action can hit 3× cold-start latency (≈3-5s total). Provisioned concurrency or keep-warm pings are worth considering for production.



Plan the Feedback Loop from Day One

`recovery_history` was added after the fact and required retrofitting the State Updater Lambda. First-class feedback design from the start makes the whole system easier to evaluate.

Where to Take This Further



Multi-Channel Recovery

Extend recovery_action to send SMS via Amazon SNS or push notifications via customer.push_token. The decision matrix already has preferred_channel per customer.



Adaptive Decision Matrix

Feed recovery outcomes (opened / converted / ignored) back into the matrix. Use a fine-tuned model to adjust discount thresholds over time — a self-improving agent.



Internationalisation

customer_profiles already contains locale and timezone. Use these to localise email content, currency formatting, and send-time optimisation.



A/B Testing Engine

Route 20% of carts to a challenger matrix. Log recovery_history with matrix_version field. Compare conversion rates in Kibana to validate rule changes before rollout.



Enhanced Fraud ML

Replace the binary fraud_risk flag with a continuous ML score from Amazon Fraud Detector. Feed the score into the decision matrix for graduated responses.

Thank you

Building AI Agents with AWS & Elasticsearch



EventBridge
Event routing



Elasticsearch
Orchestration + AI



MCP Tools
Lambda as AI tools



GitHub Repo

<https://github.com/aashanashanu/ai-abandoned-cart-recovery-agent>